

Applied Machine Learning

Unit 05 — Lecture 09 Notes

Maximum Likelihood, Model Adequacy and Over-fitting

Tofik Ali

Autumn 2026

Read this first

These notes are written so that you can learn the material *without* having been in the lecture. Every concept is developed from scratch, every number you see was produced by running the code shown beside it, and every figure was generated from real computation rather than drawn by hand.

The previous lecture produced an estimator. It set up simple linear regression, minimised the sum of squared errors, and arrived at the closed form $b_1 = S_{xy}/S_{xx}$, $b_0 = \bar{y} - b_1\bar{x}$. It did not say *why* the sum of squares.

This lecture answers that, and the answer is the central result here: **write down a Gaussian probability model for the noise, form the likelihood, take logarithms, and maximising the likelihood turns out to be identical to minimising the sum of squared errors.** Least squares is not a convenient guess. It is the maximum likelihood estimate under a specific, checkable assumption — and once you know which assumption, you know what to check.

The rest of the lecture is that checking. R^2 in three lines of arithmetic, and then two facts about it that decide whether you can trust a reported number: it **never decreases** when you add a predictor, even a column of pure noise, and on held-out data it can be **negative without limit**. Then residual plots, which see everything R^2 cannot. Then over-fitting — where training error falls at every step while test error turns up — and cross-validation, which detects the turn without ever opening the test set.

The single result to carry out of the lecture: on 30 rows, 27 **columns of pure noise raised R^2 from 0.2413 to 0.9401 and it never decreased once.** Work through Section 4.4 with a Python session open until you can say exactly why that had to happen.

Contents

1	How to use these notes	2
1.1	What you need	3
1.2	Learning outcomes	4
2	The direct regression method	4
2.1	“Closest” is a choice	4
2.2	Worked: five points, three answers	5
2.3	The relationship between the two least-squares slopes	5
3	Maximum likelihood estimation	6
3.1	The question that least squares does not answer	6
3.2	Step 0: the probability model	6

3.3	Step 1: write the likelihood	7
3.4	Step 2: take logarithms — the central result	7
3.5	Verifying the equivalence numerically	8
3.6	Step 3: the maximum likelihood estimate of σ^2	9
3.7	And this estimate is biased	9
3.8	The bias is real: measuring it	10
3.9	Change the noise model, and the estimator changes	11
4	The coefficient of determination	12
4.1	What R^2 compares	12
4.2	Worked: the whole calculation on five rows	12
4.3	Three identities worth knowing	13
4.4	R^2 never decreases when you add a predictor	13
4.5	Adjusted R^2 , and how far it gets you	14
4.6	R^2 can be negative	15
5	Checking model adequacy	16
5.1	What one number cannot see	16
5.2	Anscombe’s quartet	16
5.3	The residual plot gives four different diagnoses	17
5.4	Non-constant variance, measured	18
5.5	The normal Q–Q plot	19
5.6	A checklist you can run in five lines	20
6	Over-fitting and its detection	20
6.1	What over-fitting is	20
6.2	The degree curve	20
6.3	Over-fitting depends on n , not just on the model	21
6.4	It happens on real data too	22
7	Cross-validation as the detector	22
7.1	The idea	22
7.2	Cross-validation finds the turn	23
7.3	Why not just one validation split?	23
7.4	Choosing k	24
8	Summary	25
9	Practice questions	26
10	Solutions	27
11	References and further reading	30

1 How to use these notes

Read in order. Section 3 is the mathematical core and everything after it depends on it; Section 4 assumes the normal equations of Lecture 8; Section 7 is the payoff for Section 6. The mathematics used is one product, one logarithm, two partial derivatives and one expectation. If you can differentiate $\log \sigma^2$ you have everything you need.

There are four kinds of box. **Key idea** — the one sentence to remember a month from now. **Worked example** — a calculation done by hand, with small numbers, before any library is used. **Caution** — the specific mistake that section exists to prevent. **Try it yourself** — something to run, with the expected output given so you can check.

Code appears in a framed block, and directly beneath it a shaded block showing what that code *actually printed*. If you run it and get something different, trust your machine and tell me.

One set of five points runs through the whole document:

$$x = (1, 2, 3, 4, 5), \quad y = (2, 4, 5, 4, 5).$$

It appears in Section 2 for the three regression methods, in Section 3 for the likelihood, and in Section 4.2 for R^2 . The numbers were chosen so that every intermediate quantity is exact: $\bar{x} = 3$, $\bar{y} = 4$, $S_{xx} = 10$, $S_{yy} = 6$, $S_{xy} = 6$, $b_1 = 0.6$, $b_0 = 2.2$, $\text{SSE} = 2.4$, $R^2 = 0.6$.

1.1 What you need

```
pip install numpy scipy scikit-learn matplotlib
```

Every number in these notes was produced with:

```
import numpy, scipy, sklearn, matplotlib
print("numpy", numpy.__version__, "| scipy", scipy.__version__,
      "| sklearn", sklearn.__version__,
      "| matplotlib", matplotlib.__version__)
```

```
numpy 2.2.6 | scipy 1.15.3 | sklearn 1.7.2 | matplotlib 3.10.9
```

The only real dataset used is `load_diabetes`, which is bundled with scikit-learn and **downloads nothing**. Everything else is either Anscombe’s published quartet, typed out in full below, or a fixed-seed synthetic problem.

```
from sklearn.datasets import load_diabetes
dia = load_diabetes()
Xd, yd = dia.data, dia.target.astype(float)
print(Xd.shape)
```

```
load_diabetes: 442 rows, 10 real features
```

Do not reach for `fetch_*`

Every `fetch_*` loader downloads over HTTPS, and on the campus network those downloads fail with a certificate error you cannot fix from your machine. Nothing in this course depends on one: use the bundled loaders (`load_diabetes`, `load_breast_cancer`, `load_iris`, `load_wine`, `load_digits`) or a fixed-seed synthetic set.

Three kinds of seed — the convention for this lecture

This lecture runs more simulations than any so far, so it is worth being explicit about where the randomness lives. There are **three** kinds of seed here and they do different jobs.

1. **The lecture seed, 0.** Every arbitrary draw — the 20 000 replications of the bias simulation, the Laplace noise, the pure-noise columns, the heteroscedastic data, the Q–Q samples, the shuffled folds — is made from `np.random.default_rng(0)` or with `random_state=0`. Nothing is special about 0; what matters is that there is only one of them and that every listing shows it.
2. **The data seed, 3.** The 25 points from $y = \sin 2\pi x + \varepsilon$ that the whole over-fitting half of the lecture is about are fixed by `np.random.default_rng(3)`. That seed does not choose an outcome, it *chooses the dataset* — so it is named separately and left alone, and every section that refers to “the same 25 points” really is using the same 25 points.
3. **The split sweep, 0 to 9.** In Section 7.3 ten different validation splits are run deliberately, because *the disagreement between them is the result*. Collapsing those to one seed would delete the demonstration.

Every number in these notes reproduces *exactly* if you run the code as written. If you get something different, the difference is in your code or your library versions, not in chance.

1.2 Learning outcomes

By the end of this lecture you should be able to:

1. State what the **direct regression method** minimises, contrast it with inverse and orthogonal regression, and compute all three slopes on a small dataset by hand (CO2).
2. **Derive** the maximum likelihood estimator for simple linear regression under Gaussian noise from a blank page: state the model, write the likelihood, take logarithms, and show that maximising it is identical to minimising the sum of squared errors (CO2, CO3).
3. Derive $\hat{\sigma}_{\text{ML}}^2 = \text{SSE}/n$, explain why it is biased downwards by the factor $(n - 2)/n$, and give the unbiased alternative (CO3).
4. Compute R^2 by hand from SS_{tot} and SS_{res} , verify it against r^2 and $b_1^2 S_{xx}$, and state the two properties that make a reported R^2 untrustworthy: monotonicity in the number of predictors, and unboundedness below on held-out data (CO3).
5. Check the adequacy of a fitted model from its residuals — linearity, constant variance, normality, independence and leverage — and say for each violation what it does and does not invalidate (CO3).
6. Recognise over-fitting from the divergence of training and held-out error curves, state the degree at which the turn occurs for a given problem, and explain why the turn depends on n as well as on the model (CO2, CO3).
7. Use k -fold and leave-one-out cross-validation to detect over-fitting and select a model without opening the test set, and explain why a single validation split is not enough (CO2).

2 The direct regression method

2.1 “Closest” is a choice

Lecture 8 fitted a line by minimising $\sum_i (y_i - b_0 - b_1 x_i)^2$. That quantity is the sum of squared *vertical* distances from the points to the line, and choosing it is a decision, not a fact about the data. Three classical choices exist, and the regression literature names them.

- **Direct regression** (also *ordinary least squares*, or *the regression of y on x*) minimises the sum of squared **vertical** distances. It treats x as known exactly and places all the error in y . Slope $b_1 = S_{xy}/S_{xx}$.
- **Inverse regression** minimises the sum of squared **horizontal** distances, i.e. it regresses x on y and then re-expresses the fitted relation as y against x . Slope, so expressed, S_{yy}/S_{xy} .
- **Orthogonal regression** (also *total least squares* or *major-axis regression*) minimises the sum of squared **perpendicular** distances. Its slope comes from the leading eigenvector of the sample covariance matrix, so it is symmetric in the two variables.

Key idea

The direct method is what this course, and every examination question in it, means by “regression”. It is the right choice when x is measured without error and the task is to *predict y* from x . That asymmetry is a modelling assumption, and Section 3 turns it into a probability statement.

2.2 Worked: five points, three answers

Three defensible lines through the same five points

Take $x = (1, 2, 3, 4, 5)$ and $y = (2, 4, 5, 4, 5)$. Then $\bar{x} = 3$, $\bar{y} = 4$, and

$$S_{xx} = 4 + 1 + 0 + 1 + 4 = 10, \quad S_{yy} = 4 + 0 + 1 + 0 + 1 = 6, \quad S_{xy} = 4 + 0 + 0 + 0 + 2 = 6.$$

Direct. $b_1 = S_{xy}/S_{xx} = 6/10 = 0.6$, and $b_0 = 4 - 0.6 \times 3 = 2.2$.

Inverse. Regressing x on y gives slope $S_{xy}/S_{yy} = 6/6 = 1$; inverting, the line expressed as y against x has slope $S_{yy}/S_{xy} = 1.0$ and intercept $4 - 1 \times 3 = 1.0$.

Orthogonal. The sample covariance matrix (with $n - 1 = 4$) is

$$S = \frac{1}{4} \begin{pmatrix} 10 & 6 \\ 6 & 6 \end{pmatrix} = \begin{pmatrix} 2.5 & 1.5 \\ 1.5 & 1.5 \end{pmatrix}, \quad \lambda = \frac{4 \pm \sqrt{16 - 6}}{2} = 3.5811, 0.4189.$$

The leading eigenvector satisfies $(2.5 - 3.5811)v_1 + 1.5v_2 = 0$, so $v_2/v_1 = 0.7208$, which is the slope.

All three lines pass through $(\bar{x}, \bar{y}) = (3, 4)$ — that is forced by the intercept equation, whichever criterion you used — and they differ only in how they pivot about it.

```
S = np.cov(np.vstack([x, y])) # 2x2 sample covariance, ddof=1
lam, V = np.linalg.eigh(S)
v = V[:, np.argmax(lam)]
print(Sxy / Sxx, Syy / Sxy, v[1] / v[0])
```

```
direct      (vertical distances) slope 0.6000  intercept 2.2000
inverse     (horizontal distances) slope 1.0000  intercept 1.0000
orthogonal  (perpendicular)      slope 0.7208  intercept 1.8377
covariance matrix [[2.5, 1.5], [1.5, 1.5]]  eigenvalues [3.5811 0.4189]
r = 0.7746      r^2 = 0.6000    direct/inverse slope ratio = 0.6000
```

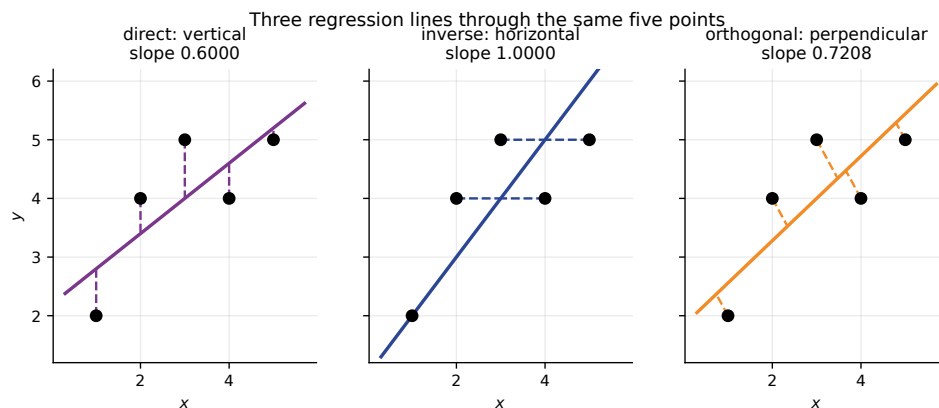


Figure 1: The three methods on the same five points. The dashed segments are the distances each method squares and adds. Every line passes through $(\bar{x}, \bar{y}) = (3, 4)$.

2.3 The relationship between the two least-squares slopes

The direct and inverse slopes are not independent. Writing $b^{\text{dir}} = S_{xy}/S_{xx}$ and $b^{\text{inv}} = S_{yy}/S_{xy}$,

$$\frac{b^{\text{dir}}}{b^{\text{inv}}} = \frac{S_{xy}}{S_{xx}} \cdot \frac{S_{xy}}{S_{yy}} = \frac{S_{xy}^2}{S_{xx}S_{yy}} = r^2.$$

On our five points, $0.6/1.0 = 0.6 = r^2$. Two consequences worth stating. First, the two slopes coincide *only* when $r^2 = 1$, i.e. when the points are exactly collinear. Second, for positive

correlation the direct slope is always the shallower of the two, and the orthogonal slope always lies between them: $0.6000 < 0.7208 < 1.0000$.

Do not use the direct fit backwards

If you fit y on x and then rearrange the equation to predict x from y , you have *not* performed inverse regression. You have used the wrong estimator: the correct inverse slope is 1.0, and inverting the direct equation gives $1/0.6 = 1.667$. The gap is the factor $1/r^2$, and it is large exactly when the fit is weak.

Each method minimises its own criterion and no other. This is worth measuring rather than asserting.

```
for name, s in [("direct", 0.6), ("inverse", 1.0), ("orthogonal", 0.7208)]:
    a = ybar - s * xbar
    r = y - (a + s * x)
    print(name, (r**2).sum(), (r**2).sum() / (1 + s*s), (r**2).sum() / (s*s))
```

	vertical SS	perpendicular SS	horizontal SS
direct	2.4000	1.7647	6.6667
inverse	4.0000	2.0000	4.0000
orthogonal	2.5458	1.6754	4.9006

Read down the diagonal: 2.4000, 4.0000, 1.6754. Each is the smallest number in its own column and in no other. There is no “best line” — there is a best line for a stated criterion. From here on, “regression” means the direct method, and the next section explains why that criterion in particular.

3 Maximum likelihood estimation

3.1 The question that least squares does not answer

Why squares? The usual answers — it is smooth, it has a closed form, it punishes large errors — are all true and all equally true of $\sum e_i^4$, which has none of the properties we actually rely on. The real answer requires a change of question. Instead of asking “which line is closest?”, ask:

*Suppose the data was generated by some random process with unknown parameters.
Which parameter values make the data I actually observed most probable?*

That is the **maximum likelihood** principle, and it is the single most important idea in this unit. It will reappear unchanged in Lecture 10 for logistic regression, where it produces cross-entropy rather than squares, and again in Unit VI for naïve Bayes.

3.2 Step 0: the probability model

The model is one line of algebra and three assumptions:

$$y_i = \beta_0 + \beta_1 x_i + \varepsilon_i, \quad \varepsilon_i \sim \mathcal{N}(0, \sigma^2) \quad \text{independently, for } i = 1, \dots, n.$$

Equivalently, $y_i \sim \mathcal{N}(\beta_0 + \beta_1 x_i, \sigma^2)$. The three assumptions, each of which Section 5 shows you how to check:

1. **Linearity.** The *mean* of y at a given x is $\beta_0 + \beta_1 x$. Not the value — the mean.
2. **Homoscedasticity.** The variance σ^2 is the *same* at every x .
3. **Independence and normality.** The ε_i are independent draws from a Normal distribution.

Note what is *not* assumed: nothing about the distribution of x . The x_i are treated as fixed, known constants. That is the asymmetry of the direct method, now stated as probability.

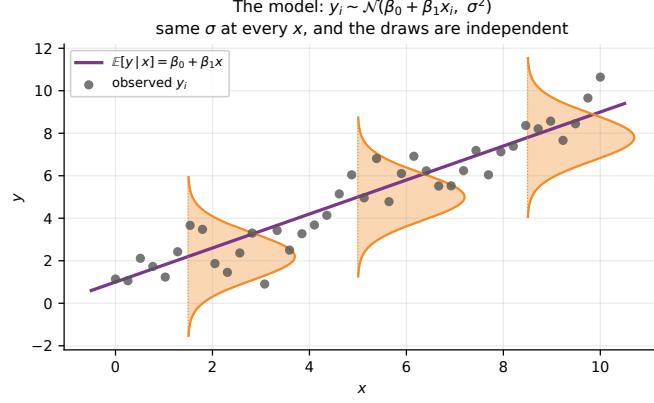


Figure 2: The line is the conditional mean; the orange bells are the distribution of y around it. Same width at every x (assumption 2), and each observation is one independent draw from its own bell.

3.3 Step 1: write the likelihood

Write $e_i = y_i - \beta_0 - \beta_1 x_i$ for the error of observation i at a candidate parameter value. Because ε_i is Normal with mean zero and variance σ^2 , the density of the observation y_i is

$$p(y_i | x_i, \beta_0, \beta_1, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{e_i^2}{2\sigma^2}\right).$$

The n observations are independent, so their joint density is the product of the individual densities. Read as a function of the *parameters* with the data held fixed — which is the reversal that gives the method its name — that product is the **likelihood**:

$$L(\beta_0, \beta_1, \sigma^2) = \prod_{i=1}^n \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{e_i^2}{2\sigma^2}\right) = (2\pi\sigma^2)^{-n/2} \exp\left(-\frac{1}{2\sigma^2} \sum_{i=1}^n e_i^2\right). \quad (1)$$

The second equality is only bookkeeping: n copies of $(2\pi\sigma^2)^{-1/2}$ multiply to $(2\pi\sigma^2)^{-n/2}$, and a product of exponentials is the exponential of a sum. But look at what that sum is. The sum of squared errors has appeared inside the exponent, and we have not chosen anything yet.

3.4 Step 2: take logarithms — the central result

$\log$ is strictly increasing, so $\arg \max L = \arg \max \log L$: taking logarithms cannot move the maximiser, and it turns the product into a sum. Write $\ell = \log L$:

$$\ell(\beta_0, \beta_1, \sigma^2) = -\frac{n}{2} \log(2\pi) - \frac{n}{2} \log \sigma^2 - \underbrace{\frac{1}{2\sigma^2} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2}_{S(\beta_0, \beta_1)}. \quad (2)$$

Now read Equation (2) as a function of β_0 and β_1 alone, with σ^2 held at any fixed positive value.

- The first term, $-\frac{n}{2} \log(2\pi)$, is a constant.
- The second term, $-\frac{n}{2} \log \sigma^2$, does not involve β_0 or β_1 either.
- The third term is $-\frac{1}{2\sigma^2} S(\beta_0, \beta_1)$, where $1/(2\sigma^2) > 0$.

So ℓ is a positive constant times $-S$, plus a constant. Maximising a negative multiple of S is minimising S . Therefore, for every fixed $\sigma^2 > 0$,

$$\arg \max_{\beta_0, \beta_1} \ell(\beta_0, \beta_1, \sigma^2) = \arg \min_{\beta_0, \beta_1} S(\beta_0, \beta_1) = \arg \min_{\beta_0, \beta_1} \text{SSE}. \quad (3)$$

The central result of this lecture

Under the assumption that the errors are independent Gaussian draws with a common variance, **maximising the likelihood is exactly the same optimisation problem as minimising the sum of squared errors**. Least squares is therefore not an arbitrary convenience — it is the maximum likelihood estimator for a specific, checkable noise model. And because σ^2 appears only as a positive multiplier of S , the value of σ^2 cannot change which line wins.

Setting the derivatives of S to zero now reproduces Lecture 8 exactly:

$$\frac{\partial S}{\partial \beta_0} = -2 \sum_i e_i = 0, \quad \frac{\partial S}{\partial \beta_1} = -2 \sum_i x_i e_i = 0,$$

which are the normal equations, with solution $\hat{\beta}_1 = S_{xy}/S_{xx}$ and $\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$. **The estimator did not move. Its justification did**, and with it the list of things you now have to check.

Two conclusions people wrongly draw from this

First, this does *not* say that your errors are Gaussian. It says least squares is the ML estimator *if* they are. Section 3.9 shows a case where they are not and the ML estimator is something else entirely. **Second**, it does not say least squares is bad when the errors are not Gaussian: by the Gauss–Markov theorem it remains the best *linear unbiased* estimator under much weaker conditions. What you lose is the optimality among *all* estimators, and every p -value.

3.5 Verifying the equivalence numerically

The claim is that the two curves — SSE against b_1 , and ℓ against b_1 — have their extrema in the same place, at every σ^2 . Take the five points and check.

```
def loglik(b0, b1, s2):
    r = y - (b0 + b1 * x)
    return (-n/2*np.log(2*np.pi) - n/2*np.log(s2) - (r**2).sum() / (2*s2))

for g in [0.2, 0.4, 0.5, 0.6, 0.7, 0.8, 1.0]:
    a = ybar - g * xbar
    print(g, ((y - (a + g*x))**2).sum(), loglik(a, g, 0.48), loglik(a, g, 1.0))
```

b1	SSE	logL(sigma^2=0.48)	logL(sigma^2=1)
0.20	4.0000	-6.926436	-6.594693
0.40	2.8000	-5.676436	-5.994693
0.50	2.5000	-5.363936	-5.844693
0.60	2.4000	-5.259770	-5.794693
0.70	2.5000	-5.363936	-5.844693
0.80	2.8000	-5.676436	-5.994693
1.00	4.0000	-6.926436	-6.594693

argmin SSE b1 = 0.60000
argmax logL s2=0.48 b1 = 0.60000
argmax logL s2=1.00 b1 = 0.60000
closed form Sxy/Sxx b1 = 0.60000

Three things to notice. The SSE column is symmetric about $b_1 = 0.6$ — it is a parabola. The two log-likelihood columns contain *different numbers* (-5.2598 against -5.7947 at the optimum), because σ^2 changes the height and the curvature. And they peak in exactly the same place, to five decimal places, which is Equation (3) measured.

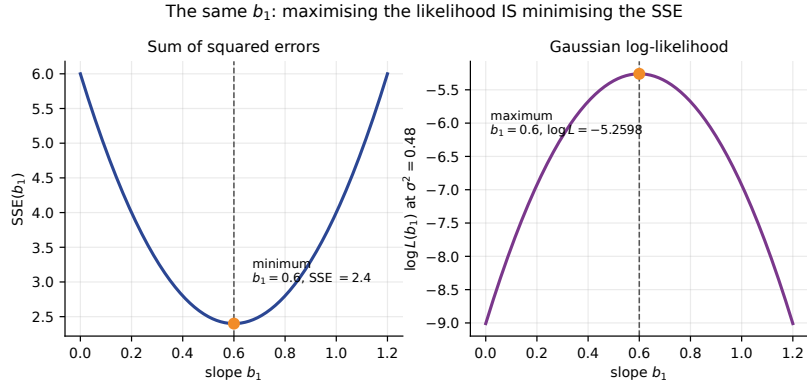


Figure 3: Left: SSE against the slope. Right: the log-likelihood at $\sigma^2 = 0.48$ — the same parabola, negated and shifted, with its maximum in the same place, because $\ell = \text{const} - \text{SSE}/(2\sigma^2)$.

3.6 Step 3: the maximum likelihood estimate of σ^2

The likelihood has a third parameter. Hold β_0, β_1 at their optimum — so that $\sum_i e_i^2 = \text{SSE}$, a fixed number — and differentiate Equation (2) with respect to σ^2 , treating σ^2 as a single symbol v :

$$\ell = \text{const} - \frac{n}{2} \log v - \frac{\text{SSE}}{2v}, \quad \frac{\partial \ell}{\partial v} = -\frac{n}{2v} + \frac{\text{SSE}}{2v^2} = 0 \quad \Rightarrow \quad v = \frac{\text{SSE}}{n}.$$

So

$$\hat{\sigma}_{\text{ML}}^2 = \frac{\text{SSE}}{n}. \quad (4)$$

The second derivative at that point is $-n^3/(2\text{SSE}^2) < 0$, so it is a maximum. Substituting Equation (4) back into Equation (2) gives the maximised log-likelihood in closed form, which is worth knowing because AIC and BIC are built from it:

$$\ell_{\text{max}} = -\frac{n}{2} (\log(2\pi\hat{\sigma}_{\text{ML}}^2) + 1). \quad (5)$$

3.7 And this estimate is biased

The slope and intercept MLEs are unbiased. The variance MLE is not.

The residuals $e_i = y_i - \hat{y}_i$ are not the errors ε_i : they have been shrunk by the act of fitting. In matrix form $\mathbf{e} = (I - H)\boldsymbol{\varepsilon}$, where H is the hat matrix and $\text{tr}(I - H) = n - p$ with p the number of estimated coefficients ($p = 2$ for simple linear regression). Taking expectations,

$$\mathbb{E}[\text{SSE}] = (n - 2) \sigma^2 \quad \Rightarrow \quad \mathbb{E}[\hat{\sigma}_{\text{ML}}^2] = \frac{n - 2}{n} \sigma^2 < \sigma^2. \quad (6)$$

The intuition is degrees of freedom. The residuals satisfy two exact linear constraints — $\sum_i e_i = 0$ and $\sum_i x_i e_i = 0$, both from the normal equations — so only $n - 2$ of them are free to vary. Dividing a sum of $n - 2$ free squares by n understates the spread.

The unbiased estimator therefore divides by the degrees of freedom:

$$s^2 = \frac{\text{SSE}}{n - 2}, \quad s = \sqrt{\frac{\text{SSE}}{n - 2}} \quad (\text{the standard error of the estimate}). \quad (7)$$

The two estimates on the five points

SSE = 2.4 and $n = 5$, so

$$\hat{\sigma}_{\text{ML}}^2 = \frac{2.4}{5} = 0.48, \quad s^2 = \frac{2.4}{3} = 0.80, \quad s = 0.8944.$$

The ratio is $(n - 2)/n = 3/5 = 0.6$: at $n = 5$ the maximum likelihood estimate of the noise variance is 40% **too small in expectation**. At $n = 100$ the same factor is 0.98 and nobody notices — but the small-sample case is exactly the case in which you needed a variance estimate.

```
sse = ((y - yhat)**2).sum()
print(sse / n, sse / (n - 2), np.sqrt(sse / (n - 2)))
print(-n/2 * (np.log(2*np.pi*(sse/n)) + 1), loglik(b0, b1, sse/n))
```

```
SSE          2.4000
sigma^2 MLE   = SSE/n      = 2.4000/5 = 0.4800
sigma^2 unbias = SSE/(n-2) = 2.4000/3 = 0.8000
ratio (n-2)/n = 0.6000      s = sqrt(unbiased) = 0.8944
maximised log-likelihood = -n/2*(log(2*pi*s2_ml)+1) = -5.2598
direct evaluation                                     = -5.2598
```

The last two lines confirm Equation (5): the closed form and the direct evaluation of Equation (2) agree to four decimals.

3.8 The bias is real: measuring it

Equation (6) is a statement about an average over datasets, so generating many datasets is the way to check it. Simulate 20,000 datasets of $n = 10$ points from a known truth with $\sigma^2 = 4$, fit each, and record both estimates.

```
rng = np.random.default_rng(0)
xs = np.linspace(0, 9, 10)
for i in range(20000):
    ys = 1.0 + 2.0*xs + rng.normal(0, 2.0, 10) # sigma^2 = 4
    b = ((xs-xs.mean())*(ys-ys.mean())).sum() / ((xs-xs.mean())**2).sum()
    e = ys - ((ys.mean() - b*xs.mean()) + b*xs)
    ml[i], unb[i] = (e**2).sum()/10, (e**2).sum()/8
```

```
true sigma^2          4.0000 (n = 10, 20000 replications)
mean of SSE/n (the MLE) 3.2117 bias -0.7883
mean of SSE/(n-2) (unbiased) 4.0146 bias +0.0146
predicted MLE mean (n-2)/n*s^2 3.2000
the MLE was too small in 73.4% of the 20000 replications
```

The theory predicted $(n - 2)/n \times 4 = 3.2000$ and the simulation returned 3.2117. That is not sampling noise: it is a systematic 20% understatement, and it occurred in nearly three runs out of four. A model that reports its own noise as smaller than it is will also report prediction intervals narrower than they should be, and confidence intervals for the coefficients that are too tight.

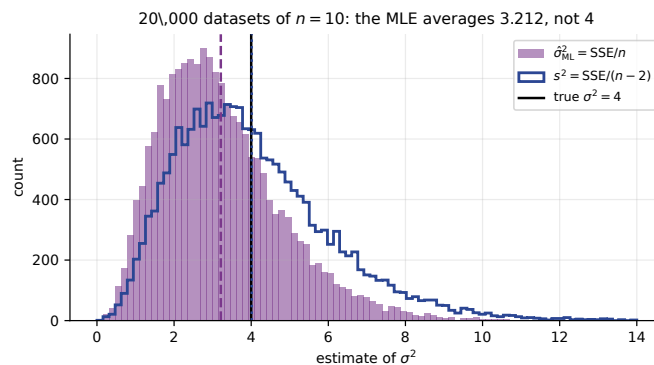


Figure 4: The same histogram stretched by $n/(n - 2) = 1.25$; both are right-skewed because SSE/σ^2 is χ_{n-2}^2 . Unbiased does not mean accurate — at $n = 10$ both estimators are wildly variable.

“Unbiased” is a statement about an average, not about your dataset

$s^2 = 4.0146$ is the mean over 20,000 datasets. On any *one* dataset of ten points the estimate ranged over an order of magnitude. Do not read unbiasedness as reliability.

3.9 Change the noise model, and the estimator changes

If maximum likelihood is what justifies least squares, then a different noise distribution must justify a different criterion. It does, and this is the sharpest way to see that the Gaussian assumption is doing real work.

Suppose ε_i follows a Laplace (double exponential) distribution with density $\frac{1}{2b} \exp(-|e|/b)$. Then

$$\ell = -n \log(2b) - \frac{1}{b} \sum_{i=1}^n |e_i|,$$

and maximising it means minimising $\sum_i |e_i|$ — **least absolute deviations**, not least squares. The recipe is the same; only the answer changes.

```
rng = np.random.default_rng(0) # the one seed for this lecture
x1 = np.linspace(0, 10, 60)
y1 = 2.0 + 1.5*x1 + rng.laplace(0, 1.0, 60); y1[13] += 25.0
ols = np.polyfit(x1, y1, 1)
lad = minimize(lambda p: np.abs(y1 - (p[0] + p[1]*x1)).sum(), [0., 1.],
               method="Nelder-Mead").x
```

```
true intercept 2.0000 true slope 1.5000
Gaussian MLE = least squares : intercept 3.0024 slope 1.3738
Laplace MLE = least abs. dev. : intercept 2.7650 slope 1.3730
sum of squared residuals OLS 622.2965 LAD 625.7886
sum of |residuals| OLS 85.8123 LAD 83.6585
```

Each estimator wins its own criterion: OLS has the smaller $\sum e_i^2$ ($622.30 < 625.79$) and LAD the smaller $\sum |e_i|$ ($83.66 < 85.81$). Neither is cheating — each is minimising exactly what it was asked to minimise, and each therefore loses on the other’s scoreboard.

Now compare them to the truth, $(\beta_0, \beta_1) = (2.0, 1.5)$. LAD recovered the intercept as 2.77 against OLS’s 3.00, so the outlier drags least squares further. The two *slopes*, though, are for practical purposes identical on this draw: 1.3730 against 1.3738. Be honest about that. A single dataset of sixty points is nowhere near enough to display an efficiency difference between two consistent estimators; what it displays is that one heavy draw moves both fits away from the truth, and moves the squared-error fit further.

What this example does and does not show

It shows that the criterion you minimise is a *modelling choice* that follows from the noise distribution you assume — Gaussian gives squares, Laplace gives absolute values — and that the two choices give genuinely different answers on the same data. It does *not* show that LAD is reliably more accurate, and one run of one dataset could not show that. If you want that claim, you need many replications, which is precisely the design of the simulation in Section 3.8. Do not over-read a single sample.

Key idea

The recipe is: state the noise distribution, write the likelihood, take logarithms, maximise. Gaussian noise gives squares. Laplace noise gives absolute values. In Lecture 10, a Bernoulli response will give cross-entropy. **Learn the recipe, not the formula it happens to produce here.**

Try it yourself

Show by hand that for a *single* parameter — fitting only a constant, $y_i = \mu + \varepsilon_i$ — the Gaussian MLE of μ is the sample mean and the Laplace MLE is the sample median. Then check numerically on $y = (1, 2, 3, 4, 100)$: you should get $\hat{\mu}_{\text{Gauss}} = 22.0$ and $\hat{\mu}_{\text{Laplace}} = 3.0$. That single number is the whole argument for robust regression.

4 The coefficient of determination

4.1 What R^2 compares

Having fitted a line, you want one number for how good it is. R^2 answers a specific comparative question: *how much better is my line than the flat line $\hat{y} = \bar{y}$?*

Start from the identity, valid whenever the model contains an intercept:

$$\underbrace{\sum_i (y_i - \bar{y})^2}_{\text{SS}_{\text{tot}}} = \underbrace{\sum_i (\hat{y}_i - \bar{y})^2}_{\text{SS}_{\text{reg}}} + \underbrace{\sum_i (y_i - \hat{y}_i)^2}_{\text{SS}_{\text{res}}}. \quad (8)$$

It follows from writing $y_i - \bar{y} = (\hat{y}_i - \bar{y}) + (y_i - \hat{y}_i)$ and squaring: the cross term is $2 \sum_i (\hat{y}_i - \bar{y}) e_i = 2(\hat{\beta}_0 \sum_i e_i + \hat{\beta}_1 \sum_i x_i e_i - \bar{y} \sum_i e_i) = 0$, because the normal equations force $\sum_i e_i = 0$ and $\sum_i x_i e_i = 0$. Then

$$R^2 = 1 - \frac{\text{SS}_{\text{res}}}{\text{SS}_{\text{tot}}} = \frac{\text{SS}_{\text{reg}}}{\text{SS}_{\text{tot}}}. \quad (9)$$

The decomposition needs the intercept

Fit without an intercept and $\sum_i e_i \neq 0$, the cross term does not vanish, Equation (8) fails, and the two expressions in Equation (9) stop agreeing. Different software then reports different “ R^2 ” for the same no-intercept fit. If you ever need a no-intercept model, report the residual standard error instead.

4.2 Worked: the whole calculation on five rows

R^2 from the definition

The five points again, with $\hat{y} = 2.2 + 0.6x$.

x	y	$\hat{y}$	$y - \bar{y}$	$(y - \bar{y})^2$	$y - \hat{y}$	$(y - \hat{y})^2$
1	2	2.8	-2.0	4.00	-0.8	0.64
2	4	3.4	0.0	0.00	0.6	0.36
3	5	4.0	1.0	1.00	1.0	1.00
4	4	4.6	0.0	0.00	-0.6	0.36
5	5	5.2	1.0	1.00	-0.2	0.04
				SS _{tot} = 6.00	SS _{res} = 2.40	

$$R^2 = 1 - \frac{2.40}{6.00} = 0.6000, \quad s = \sqrt{\frac{2.40}{5-2}} = \sqrt{0.8} = 0.8944.$$

Two independent checks. First, $\text{SS}_{\text{reg}} = 6.00 - 2.40 = 3.60$, and separately $b_1^2 S_{xx} = 0.36 \times 10 = 3.60$. Second, $r = S_{xy} / \sqrt{S_{xx} S_{yy}} = 6 / \sqrt{60} = 0.7746$ and $r^2 = 0.6000$.

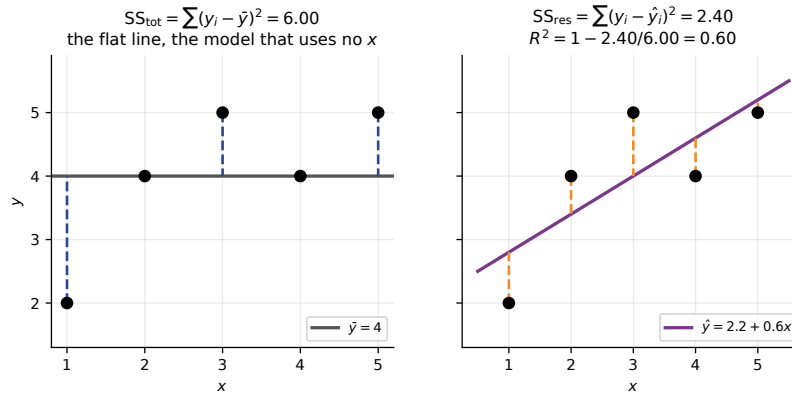


Figure 5: Left: deviations from $\bar{y} = 4$, squaring and adding to 6.00. Right: deviations from the fitted line, adding to 2.40. R^2 is the fraction of the left picture that the right picture removed.

4.3 Three identities worth knowing

For *simple* linear regression — one predictor plus an intercept — the following are all the same number:

$$R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}} = \frac{SS_{\text{reg}}}{SS_{\text{tot}}} = \frac{b_1^2 S_{xx}}{S_{yy}} = r^2 = \text{corr}(y, \hat{y})^2. \quad (10)$$

The last of these is the one that generalises: with several predictors, r^2 of any single predictor is no longer R^2 , but the squared correlation between y and the fitted values still is.

```
from sklearn.metrics import r2_score
print(r2_score(y, yhat))
print(LinearRegression().fit(x[:, None], y).score(x[:, None], y))
```

```
SSreg = SStot - SSres = 3.60      b1^2 * Sxx = 3.60
R2 = 1 - SSres/SStot = 1 - 2.40/6.00 = 0.6000
r^2 = (Sxy/sqrt(Sxx*Syy))^2 = 0.6000
sklearn r2_score                = 0.6000
s = sqrt(SSres/(n-2))            = 0.8944
```

Four routes to the same 0.6000. `r2_score` is three lines of arithmetic, not a verdict, and the rest of this section is about what those three lines cannot see.

4.4 R^2 never decreases when you add a predictor

Here is the property that makes a reported R^2 untrustworthy as a measure of model quality.

Key idea

Adding a column to a linear model can never decrease its training R^2 , no matter what is in the column. The old fit is still available in the enlarged model — just set the new coefficient to zero — so the minimised SS_{res} can only stay the same or fall.

The argument is one sentence, and it is worth checking that reality obeys it. Take 30 rows with one genuine predictor, and append columns of $\mathcal{N}(0, 1)$ noise generated *without ever looking at y*.

```
rng = np.random.default_rng(0) # the one seed for this lecture
xr = rng.normal(0, 1, 30)
yr = 1.0 + 2.0*xr + rng.normal(0, 2.0, 30)
noise = rng.normal(0, 1, (30, 27))
for k in range(28):
    X = np.c_[xr, noise[:, :k]]
    print(k, LinearRegression().fit(X, yr).score(X, yr))
```

extra noise columns	p	R2	adjusted R2
---------------------	---	----	-------------

0	1	0.241296	+0.214199
1	2	0.258852	+0.203952
2	3	0.344538	+0.268908
5	6	0.402777	+0.246979
10	11	0.660344	+0.452777
15	16	0.743023	+0.426743
20	21	0.793909	+0.252920
25	26	0.852039	-0.430293
27	28	0.940085	-0.737539

R2 decreased at any step? False (28 nested models)
smallest single-step change +0.00000315
R2 with 27 pure-noise columns and 30 rows: 0.940085

R^2 went from 0.2413 to 0.9401 and *never once fell*: the smallest single step across all 27 additions was $+3 \times 10^{-6}$, on the positive side. Extrapolating, at $p = 29$ the model would have as many parameters as rows and R^2 would be exactly 1.

What this rules out

R^2 cannot be used to compare models of different sizes, and “we added features and R^2 went up” is not evidence of anything. It is a theorem.

4.5 Adjusted R^2 , and how far it gets you

The standard correction charges you for each parameter:

$$R_{\text{adj}}^2 = 1 - (1 - R^2) \frac{n - 1}{n - p - 1}, \quad (11)$$

where p is the number of predictors excluding the intercept. Equivalently, it replaces the two sums of squares by their mean squares: $1 - \frac{\text{SS}_{\text{res}}/(n - p - 1)}{\text{SS}_{\text{tot}}/(n - 1)}$. Unlike R^2 , it can fall when a useless column is added, and it can be negative.

On the 30-row experiment it worked perfectly: +0.2142 at $p = 1$ and **-0.7375** at $p = 28$. Now try the same trick on real data, where n is large.

```
rng = np.random.default_rng(0) # the one seed for this lecture
NOISE = rng.normal(0, 1, (442, 400))
for k in [0, 10, 50, 100, 200, 300, 400]:
    X = np.c_[Xd, NOISE[:, :k]]
    m = LinearRegression().fit(X, yd)
    cv = cross_val_score(LinearRegression(), X, yd,
                        cv=KFold(5, shuffle=True, random_state=0), scoring="r2")
```

load_diabetes: 442 rows, 10 real features				
noise cols	p	R2	adjusted R2	5-fold CV R2
0	10	0.517748	+0.506559	+0.489155
10	20	0.538221	+0.516283	+0.488619
50	60	0.592908	+0.528799	+0.424743
100	110	0.643486	+0.525008	+0.337363
200	210	0.759013	+0.539935	-0.105104
300	310	0.887148	+0.620092	-1.980381
400	410	0.976075	+0.659652	-5.760536

Read the last row. With 410 pure-noise columns on 442 patients, R^2 is 0.9761, adjusted R^2 is +0.6597 — *higher* than it was with no noise at all — and the cross-validated R^2 is -5.7605.

Key idea

Adjusted R^2 is a penalty, not a test. On a small dataset it caught a fabricated model completely; on a larger one it was still reporting $+0.54$ while the model's actual predictive value was catastrophically negative. **Report adjusted R^2 if you like; trust the number computed on rows the fit never saw.**

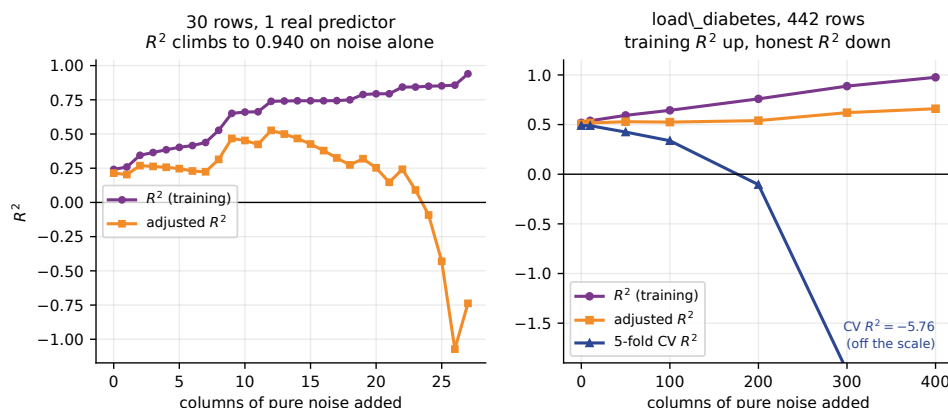


Figure 6: Left, 30 rows: R^2 climbs to 0.908 on noise while adjusted R^2 collapses to -1.66 . Right, 442 rows: R^2 climbs, adjusted R^2 stays flat and complacent near 0.5, and cross-validated R^2 falls off the bottom of the chart. Only the third curve is measuring prediction.

4.6 R^2 can be negative

$R^2 \in [0, 1]$ is only guaranteed *on the data the intercept was fitted to*. There, the flat line $\hat{y} = \bar{y}$ is one of the fits available to the model, so the minimised SS_{res} cannot exceed SS_{tot} . On held-out data no such guarantee exists: $\bar{y}_{\text{test}}$ is a number the model has never seen.

Negative R^2 in three numbers

Held-out values $y = (1, 2, 3)$; the model predicts $(3, 2, 1)$. $\bar{y}_{\text{test}} = 2$, so

$$SS_{\text{tot}} = 1 + 0 + 1 = 2, \quad SS_{\text{res}} = 4 + 0 + 4 = 8, \quad R^2 = 1 - \frac{8}{2} = -3.$$

Predicting the constant 2 for all three would have given $R^2 = 0$. There is no lower bound at all: scale the predictions up and $R^2 \rightarrow -\infty$.

```
print(r2_score([1., 2., 3.], [3., 2., 1.]))
print(r2_score([1., 2., 3.], [2., 2., 2.]))

y_test      [1. 2. 3.]
prediction   [3. 2. 1.]
mean of y_test 2.0000  SStot 2.0000  SSres 8.0000
R2 = 1 - 8.0/2.0 = -3.0000      sklearn -3.0000
R2 of the constant prediction 2.0: 0.0000
```

This is not a contrived edge case. It is the routine outcome of an over-fitted model, as the next section will show at length.

```
c = np.polynomial.Polynomial.fit(x_train, y_train, 14) # 25 training points
print(r2_score(y_train, c(x_train)), r2_score(y_test, c(x_test)))
```

degree	train R2	test R2
3	+0.894255	+0.875699
9	+0.918465	+0.838709
14	+0.953629	-1.105552
18	+0.959084	-2864.487739

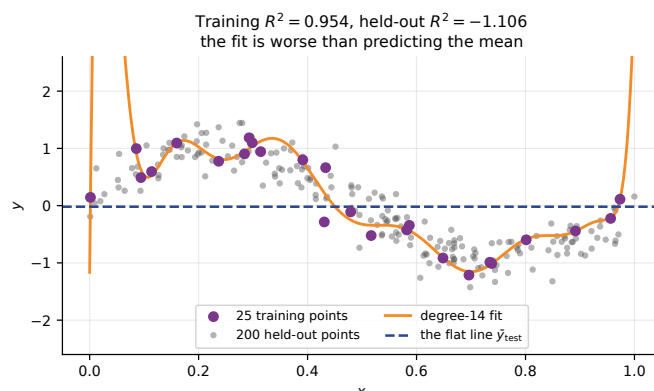


Figure 7: Training $R^2 = 0.954$, held-out $R^2 = -1.106$. The curve passes beautifully close to every training point and leaves the data entirely between them. The blue dashed line — predict the mean and go home — beats it.

Key idea

A negative held-out R^2 is not a bug and not a rounding error. It means the model is worse than a constant. Whenever you see one, the honest report is “this model has no predictive value on this data”.

5 Checking model adequacy

5.1 What one number cannot see

R^2 is a ratio of two sums. It has no access to the *order* of the residuals, their *shape*, or their *pattern against $\hat{y}$* . Every assumption of Section 3.2 can fail while R^2 stays high.

Assumption	How it fails	What it invalidates
Linearity	residuals versus fitted show an arch or a U	the coefficients themselves; the model is simply wrong
Constant variance	the residual cloud fans out with $\hat{y}$	standard errors, p -values, all intervals
Normality	Q-Q plot bends at the ends	p -values and intervals in small samples
Independence	residuals plotted in order drift or oscillate	standard errors, badly; often understates them
No single point in charge	one row has leverage near 1	everything: delete the row and the fit moves

5.2 Anscombe's quartet

In 1973 F. J. Anscombe published four datasets of eleven points each, constructed to share almost every summary statistic. Lecture 8 used them to make the point that a table of numbers is not a substitute for a plot; here we take the next step and read the *residuals*, which turn “these look different” into four separate, nameable diagnoses. The data is short enough to type out.

```
x123 = [10., 8., 13., 9., 11., 14., 6., 4., 12., 7., 5.]
y1 = [8.04, 6.95, 7.58, 8.81, 8.33, 9.96, 7.24, 4.26, 10.84, 4.82, 5.68]
```



```

y2 = [9.14, 8.14, 8.74, 8.77, 9.26, 8.10, 6.13, 3.10, 9.13, 7.26, 4.74]
y3 = [7.46, 6.77, 12.74, 7.11, 7.81, 8.84, 6.08, 5.39, 8.15, 6.42, 5.73]
x4 = [8., 8., 8., 8., 8., 8., 8., 8., 19., 8., 8., 8.]
y4 = [6.58, 5.76, 7.71, 8.84, 8.47, 7.04, 5.25, 12.50, 5.56, 7.91, 6.89]

```

set	mean x	mean y	sd x	sd y	intercept	slope	R2	SSres
I	9.000	7.501	3.317	2.032	3.0001	0.5001	0.6665	13.7627
II	9.000	7.501	3.317	2.032	3.0009	0.5000	0.6662	13.7763
III	9.000	7.500	3.317	2.030	3.0025	0.4997	0.6663	13.7562
IV	9.000	7.501	3.317	2.031	3.0017	0.4999	0.6667	13.7425

Identical means, identical standard deviations, the same fitted line $\hat{y} = 3.00 + 0.50x$, the same $R^2 = 0.666$ and the same residual sum of squares to two decimals. Anyone reading only that table would conclude these are four samples from the same population.

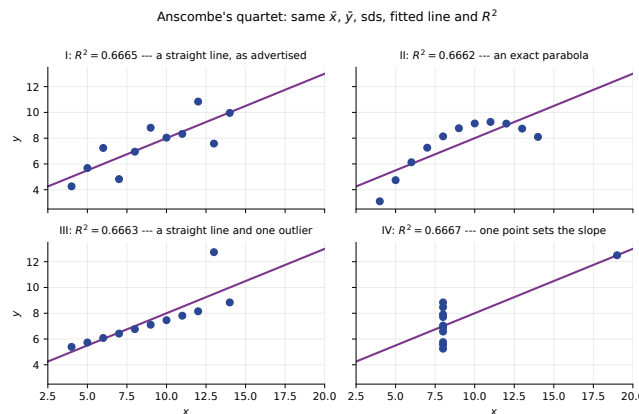


Figure 8: I is a genuine noisy line; II an exact parabola (wrong model, not noisy data); III a perfect line with one contaminated point; IV ten points at $x = 8$ and one at $x = 19$, which alone determines the slope.

5.3 The residual plot gives four different diagnoses

Plotting the residuals $e_i = y_i - \hat{y}_i$ against the fitted values $\hat{y}_i$ is the single most informative diagnostic in regression. A model that fits well produces a shapeless horizontal band. Anything else is a message.

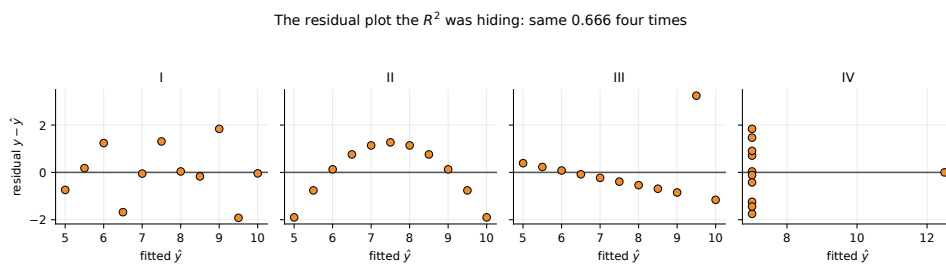


Figure 9: Residuals against fitted values for the four Anscombe datasets. Same $R^2 = 0.666$; four different pictures.

Four numeric diagnostics separate them completely. *Curvature* R^2 is what a parabola fitted to the residuals explains. *Leverage* is $h_i = 1/n + (x_i - \bar{x})^2/S_{xx}$, the influence of row i on its own fitted value; it sums to $p = 2$, so an average row has $h \approx 0.18$ here.

```

e = ya - (ic + sl * xa)
curv = r2_score(e, np.polyval(np.polyfit(xa, e, 2), xa))
h = 1/len(xa) + (xa - xa.mean())**2 / ((xa - xa.mean())**2).sum()

```

set	max e	e max/ e 2nd	curvature R2	Shapiro p	max leverage	slope w/o pt
I	1.921	1.04	0.0623	0.5456	0.3182	0.5028
II	1.901	1.00	1.0000	0.0929	0.3182	0.6267
III	3.241	2.80	0.0550	0.0016	0.3182	0.5770

IV	1.839	1.05	0.0000	0.7800	1.0000	undefined
----	-------	------	--------	--------	--------	-----------

- **I** — nothing stands out. Curvature $R^2 = 0.06$, largest residual barely bigger than the next, Shapiro $p = 0.55$, leverage 0.32. Deleting the highest-leverage point moves the slope from 0.5001 to 0.5028. This is what an adequate model looks like.
- **II** — a parabola fitted to the residuals explains 100.00% of them. There is no noise left, only unmodelled curvature: the correct model is quadratic, and the linear fit’s “residuals” are entirely signal.
- **III** — the largest residual is $2.80\times$ the next, and Shapiro rejects normality at $p = 0.0016$. Ten of the eleven points lie exactly on a line; one does not. The fitted slope of 0.4997 belongs to neither the ten nor the one.
- **IV** — leverage **1.0000**. One point supplies the entire spread of x , so it fixes its own fitted value exactly, and deleting it leaves the slope **undefined** (all remaining x are equal). There is no evidence for linearity here at all.

Key idea

The same R^2 described a good fit, a wrong model, a contaminated dataset and a fit with no evidence behind it. **Never report an R^2 without a residual plot beside it.**

5.4 Non-constant variance, measured

Heteroscedasticity — variance changing with x or $\hat{y}$ — is the commonest violation in practice, because so many real quantities have noise proportional to their size. Construct a case where the noise sd grows linearly with x .

```
rng = np.random.default_rng(0) # the one seed for this lecture
xh = np.sort(rng.uniform(1, 10, 200))
yh = 3.0 + 2.0*xh + rng.normal(0, 1, 200) * (0.25*xh) # sd grows with x
eh = yh - np.polyval(np.polyfit(xh, yh, 1), xh)
aux = LinearRegression().fit(fh.reshape(-1, 1), eh**2) # Breusch-Pagan
lm = len(xh) * aux.score(fh.reshape(-1, 1), eh**2)
```

```
fit: intercept 2.8790 slope 2.0011 R2 0.9267
residual sd, lower half of x 1.0460
residual sd, upper half of x 1.8988 ratio 1.82
Breusch-Pagan LM = n*R2(e^2 on yhat) = 22.9065 chi2(1) p = 0.000002
Shapiro-Wilk on the residuals p = 0.031073
after log(y): Breusch-Pagan LM = 3.2426 p = 0.0717 sd ratio 0.89
```

$R^2 = 0.9267$, which most people would call a good fit. The residual standard deviation over the upper half of x is $1.82\times$ that over the lower half, and the Breusch–Pagan test — regress the squared residuals on the fitted values and compare nR^2 to a χ_1^2 — returns 22.91 with $p \approx 2 \times 10^{-6}$. The variance is not constant, and no amount of R^2 was ever going to say so.

Note the fourth line honestly: the Shapiro–Wilk test also rejects, but only just, at $p = 0.031073$. Non-constant variance shows up as apparent non-normality, because a mixture of Normals with different variances is not Normal — but it shows up *weakly*, because that mixture is still symmetric and unimodal. That is the useful lesson: the test that is aimed at the actual defect ($p \approx 2 \times 10^{-6}$) is three decimal orders more decisive than the test that is not ($p = 0.031$). The two diagnostics are not independent, and the residual plot is what tells you which problem you actually have.

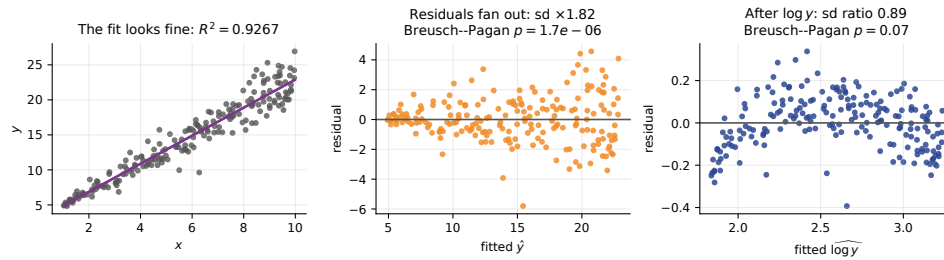


Figure 10: Left, the plot you would put in a report. Middle, the plot that tells the truth: a wedge, wide on the right. Right, the same data after $\log y$ — much closer to a shapeless band, which is what a residual plot should look like, though the transform has not removed the effect entirely.

What heteroscedasticity does and does not break

It does **not** bias the coefficients: least squares stays unbiased without any constant-variance assumption. What it breaks is everything downstream — standard errors, t and F statistics, p -values, confidence and prediction intervals — because all of them are computed as if σ were a single number. Taking $\log y$ here moved the test statistic from 22.91 to 3.24, and its p -value from 2×10^{-6} to 0.072 — a large improvement, but not a clean bill of health. A transform is a remedy to be checked, not a box to be ticked.

5.5 The normal Q–Q plot

Sort the residuals and plot them against the quantiles they would occupy if they were Gaussian. A straight line means the residuals are Normal; departures at the ends are what matter.

```
q = stats.norm.ppf((np.arange(1, n+1) - 0.5) / n)
plt.scatter(q, np.sort(e - e.mean()))
```

residual model	corr(ordered resid, normal quantile)	Shapiro p	kurtosis
Gaussian noise	0.9935	0.198668	-0.5004
$t(3)$ noise	0.9626	0.000009	2.7920
uniform noise	0.9845	0.003336	-1.0901

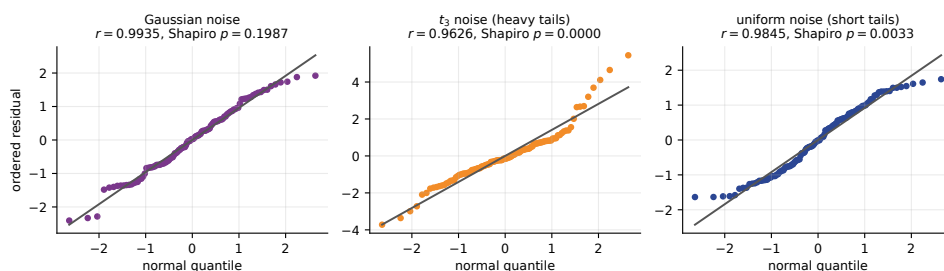


Figure 11: Three sets of 120 residuals. Gaussian sits on the line. t_3 bends *away* at both ends — heavy tails. Uniform bends *towards* the line — short tails. All three correlations exceed 0.97, which is why the picture beats the summary statistic here.

Read the shape, not the number. All three correlations are above 0.97, and yet only one of the three sets is Normal. Heavy tails show as an S-curve that falls below the line on the left and rises above it on the right; short tails show the mirror image. Neither is fatal for the coefficients, and both invalidate the p -values.

5.6 A checklist you can run in five lines

Try it yourself

For any fitted regression, produce these four plots and look at them before you report anything:

1. residuals against fitted values — linearity and constant variance;
2. a normal Q–Q plot of the residuals — normality;
3. residuals against observation order or time — independence;
4. leverage $h_i = 1/n + (x_i - \bar{x})^2/S_{xx}$ against the residual — influential points. Flag any $h_i > 2p/n$.

On the four Anscombe datasets these four plots give four different verdicts from the identical $R^2 = 0.666$. That is the point of the exercise.

6 Over-fitting and its detection

6.1 What over-fitting is

A model **over-fits** when it has started to learn the noise in the training rows — structure that is real in those rows and will not repeat. The signature is two error curves that separate: training error keeps falling, error on unseen rows stops falling and turns upward.

Why it must happen for polynomials is arithmetic. A degree- d polynomial has $d + 1$ free coefficients. Given n points with distinct x , a polynomial of degree $n - 1$ passes exactly through every one of them, noise included, and $R^2 = 1$ on the training set. Perfect training fit is a symptom, not an achievement.

The running example is 25 points drawn from $y = \sin(2\pi x) + \varepsilon$ with $\sigma = 0.25$ on $[0, 1]$. Because the truth and the noise level are known, the experiment can be graded: no model can do better than an RMSE of 0.25 on fresh data.

25 points from $y = \sin 2\pi x + \varepsilon$: the same data, four models

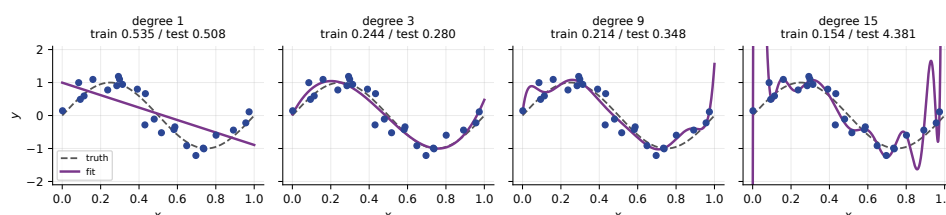


Figure 12: The same 25 points, four models. Degree 1 cannot bend (*under-fitting*); degree 3 tracks the truth; degree 9 ripples; degree 15 passes near every point and leaves the dashed truth entirely.

6.2 The degree curve

```
rng = np.random.default_rng(3) # DATA seed: fixes these 25 points
x_train = np.sort(rng.uniform(0, 1, 25))
y_train = np.sin(2*np.pi*x_train) + rng.normal(0, 0.25, 25)
x_test = np.sort(rng.uniform(0, 1, 2000))
y_test = np.sin(2*np.pi*x_test) + rng.normal(0, 0.25, 2000)
for d in range(21):
    c = np.polynomial.Polynomial.fit(x_train, y_train, d)
    tr = np.sqrt(np.mean((y_train - c(x_train))**2))
    te = np.sqrt(np.mean((y_test - c(x_test))**2)) # 2000 fresh rows
```

degree	params	train RMSE	test RMSE	
0	1	0.7507	0.7523	
1	2	0.5347	0.5084	
2	3	0.5290	0.5147	
3	4	0.2441	0.2795	<-- test minimum
4	5	0.2433	0.2820	
5	6	0.2273	0.2828	
8	9	0.2255	0.2842	
9	10	0.2143	0.3484	
12	13	0.1843	0.5966	
13	14	0.1617	1.4545	
15	16	0.1540	4.3811	
18	19	0.1518	36.7111	
19	20	0.1473	533.7692	
20	21	0.1466	153.1554	

train RMSE strictly non-increasing? True largest increase $-3.29\text{e-}05$
 test minimum at degree 3, RMSE 0.2795 (noise floor 0.2500)
 degree 20: train RMSE 0.1466 (60.1% of degree-3), test RMSE 153.1554 (547.9x worse)

Three readings.

The training curve is monotone, and this was checked, not assumed. Every one of the twenty steps reduced the training RMSE, from 0.7507 to 0.1466; the largest single change was -3.29×10^{-5} , i.e. negative. This is the same theorem as Section 4.4: a degree- $(d+1)$ model contains every degree- d model.

The test curve turns at degree 3. Its minimum, 0.2795, is a whisker above the noise floor of 0.2500, which is the best any model could do. After that it rises, slowly at first and then not slowly: 0.35 at degree 9, 4.38 at degree 15, 153 at degree 20.

Between degree 3 and degree 20, the training score improved by 40% and the model became unusable — 548 times worse on fresh data. If your only instrument is the training score, that entire journey looks like progress.

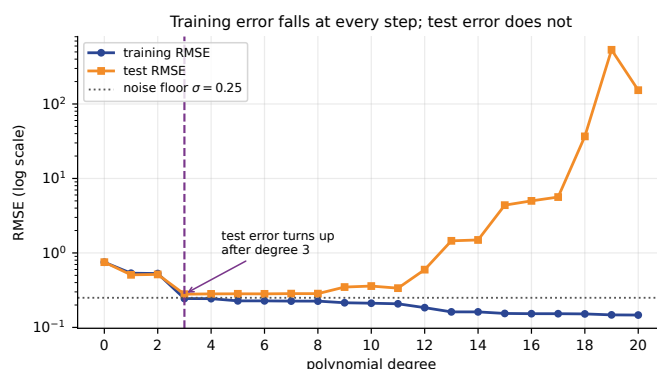


Figure 13: Log scale on the vertical axis, or the right-hand end would not fit on the page. The blue curve is monotone by construction; the orange curve owes you nothing, and after degree 3 it stops paying.

Read the tail of the table honestly

Degree 19 has a larger test RMSE (533.8) than degree 20 (153.2). The curve is not monotone once the design matrix is badly conditioned: past that point the numbers are dominated by numerical behaviour and by which particular 25 rows we drew. The teaching content is the *turn at degree 3*, not the ordering of the last few entries.

6.3 Over-fitting depends on n , not just on the model

Change only the number of training rows and rerun.

n_train	best degree	test RMSE at best	test RMSE at degree 20
15	3	0.2959	13292.3268
25	5	0.3102	2867.4761
50	6	0.2634	0.3256
200	5	0.2547	0.2656
1000	7	0.2536	0.2572

At $n = 15$, degree 20 produces a test RMSE of thirteen thousand. At $n = 1000$, the *same* twenty-one-parameter model gives 0.2572 against a best of 0.2536 and a noise floor of 0.2500 — it costs 0.0036, about one and a half per cent.

Key idea

Over-fitting is not a property of the model. It is a property of the model *and* the amount of data. “How complex a model may I use?” has no answer until you say n .

6.4 It happens on real data too

```
Xtr, Xte, ytr, yte = train_test_split(Xd, yd, test_size=0.25, random_state=0)
for deg in (1, 2, 3):
    pipe = make_pipeline(PolynomialFeatures(deg, include_bias=False),
                          LinearRegression()).fit(Xtr, ytr)
    print(pipe.score(Xtr, ytr), cross_val_score(pipe, Xtr, ytr, cv=5).mean(),
          pipe.score(Xte, yte))
```

model	train R2	5-fold CV R2	held-out R2	params
linear (10 features)	+0.5554	+0.5224	+0.3594	10
degree-2 polynomial	+0.6469	+0.3827	+0.2440	65
degree-3 polynomial	+0.9179	-1212.1828	-22.0428	285
predicting the training mean for every patient: held-out R2 -0.0001				

285 parameters against 331 training rows. The training column goes *up* at every row of the table and the held-out column goes *down* at every row. Reading only the training column, you would ship the worst of the three models. And the last line supplies the yardstick: predicting a constant scores -0.0001 on the held-out rows, so the degree-3 model at -22.04 is more than twenty times worse than doing nothing at all.

7 Cross-validation as the detector

7.1 The idea

The over-fitting section detected the turn using a test set. That is not a method you can use during model development, because a test set that has been consulted repeatedly is no longer a test set — every choice you make from it leaks information about it into your model.

k -fold cross-validation, introduced with the splitting strategies of Unit IV, solves this using only the training rows. Split them into k roughly equal parts (folds). For each fold in turn, fit the model on the other $k - 1$ folds and score it on the held-out fold. Average the k scores.

Every row is used for testing exactly once and for training $k - 1$ times. You obtain a held-out estimate without spending a held-out set, and because you get k scores rather than one, you also get a spread. The cost is k model fits. The extreme case $k = n$ is **leave-one-out cross-validation** (LOOCV): n fits, each on $n - 1$ rows, and no randomness at all, because there is only one way to leave one row out.

The test set is not part of this

The test set is opened once, at the end, to report a number — never to choose a degree, a scaler or a threshold. Cross-validation happens entirely inside the training rows. If you have used the test set to make a decision, whatever you report from it is a training score wearing a disguise.

7.2 Cross-validation finds the turn

```
from sklearn.model_selection import KFold, cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import PolynomialFeatures

kf = KFold(5, shuffle=True, random_state=0)
for d in range(16):
    pipe = make_pipeline(PolynomialFeatures(d), LinearRegression())
    cv = -cross_val_score(pipe, x_train[:, None], y_train, cv=kf,
                          scoring="neg_root_mean_squared_error").mean()
```

degree	train RMSE	5-fold CV RMSE	LOOCV RMSE	test RMSE
0	0.7507	0.7769	0.7819	0.7523
1	0.5347	0.6319	0.5895	0.5084
2	0.5290	0.9097	0.6396	0.5147
3	0.2441	0.3239	0.2799	0.2795
4	0.2433	0.6234	0.3163	0.2820
5	0.2273	0.6711	0.2912	0.2828
6	0.2273	1.3741	0.4295	0.2823
7	0.2257	4.6874	0.5433	0.2843
8	0.2255	3.1131	1.3879	0.2842
9	0.2143	32.7952	0.4516	0.3484
10	0.2114	143.9583	2.8380	0.3592
11	0.2076	710.1122	17.7364	0.3367
12	0.1843	661.0253	23.7564	0.5966
13	0.1617	1959.3824	13.9714	1.4545
14	0.1616	7218.1668	73.5354	1.4941
15	0.1540	23379.8414	49.8944	4.3811

train RMSE picks degree 15 (the largest offered)

5-fold CV picks degree 3

LOOCV picks degree 3

the test set says 3

The training column picks the largest model on offer, as it always will. Both cross-validated columns pick degree 3 — the same answer the test set gives — **using nothing but the 25 training rows**. That single table is the argument for cross-validation.

Note that the CV columns are much larger than the test column at high degree. That is expected and it is not a defect: each CV fit uses only 20 of the 25 rows, so its polynomial is even less constrained than the one fitted to all 25. Cross-validation is a slightly *pessimistic* estimate of the error of the final model, and pessimism in this direction is exactly what you want from a detector.

7.3 Why not just one validation split?

The obvious cheaper alternative is to hold out one chunk of the training rows and use it to choose. It works, on average, and the variance is the problem.

```
for seed in range(10):
    xa, xb, ya, yb = train_test_split(x_train, y_train, test_size=0.4,
                                      random_state=seed)
    curve = [rmse(yb, Polynomial.fit(xa, ya, d)(xb)) for d in range(11)]
```

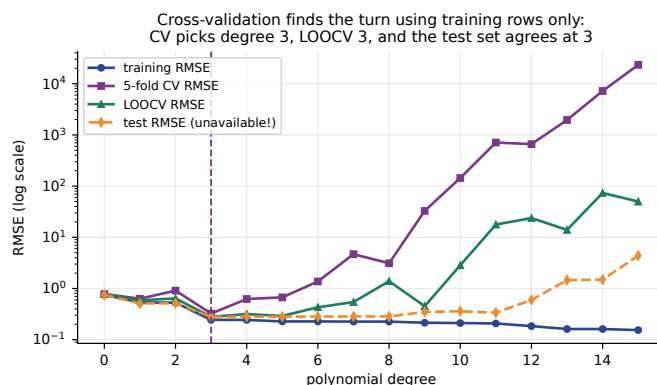


Figure 14: Four curves. The orange dashed one is the test error, drawn here only because this is a demonstration; in a real project you may not look at it. The purple and green curves reproduce its minimum without it.

```
print(int(np.argmin(curve)))
```

```
ten different single validation splits of the same 25 rows
degree chosen by each split: [3, 3, 3, 3, 4, 5, 3, 3, 3, 3]
chosen degrees range from 3 to 5
spread of the estimated RMSE at degree 3: 0.2243 to 0.4415
5-fold CV RMSE at degree 3 (one number, no split lottery): 0.3239
```

Seven of the ten splits chose degree 3 and three did not. Worse, the *estimate* of the error at degree 3 ranged from 0.2243 to 0.4415 — a factor of two — from nothing but which rows happened to land in which half. Report the first and you look good; report the second and you look bad; neither is a property of the model.

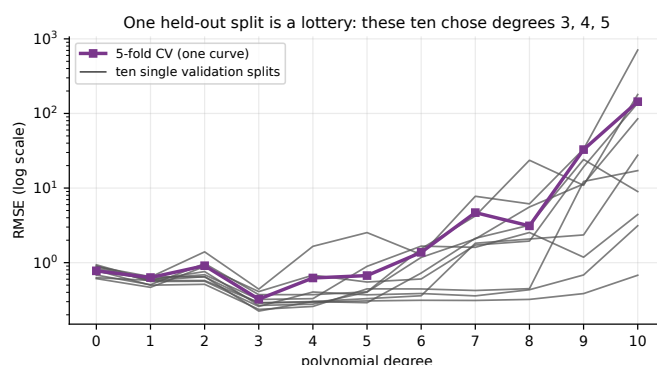


Figure 15: Ten single validation splits of the same 25 rows in grey, and the 5-fold CV curve in purple. k -fold averages the lottery away and uses every row for training.

7.4 Choosing k

	Bias of the estimate	Variance and cost
$k = 2$	pessimistic: each fit sees only half the data	cheap, two fits, but noisy
$k = 5$ or 10	small; the usual default	moderate: five or ten fits
$k = n$ (LOOCV)	almost none: each fit sees $n - 1$ rows	n fits, and the training sets overlap almost completely, so the n scores are highly correlated

In the table of Section 7.2, LOOCV tracked the test curve more closely at small degree — 0.2799 against a test 0.2795 at degree 3 — because each of its fits saw 24 rows rather than 20. It cost

25 fits against 5. On 25 rows that is free; on 250,000 rows it is not.

The rule that outranks the choice of k

Everything the model learns must be refitted inside every fold. A polynomial degree, a scaler, a feature selection, a decision threshold. If a choice was made using all the rows and then cross-validated, the resulting score is not a held-out score. **Pipeline** exists precisely so that doing this correctly is easier than doing it wrongly, which is why every listing in this section wraps its preprocessing in one.

Try it yourself

Re-run the degree sweep with `KFold(5, shuffle=True, random_state=1)` instead of `random_state=0`. The individual CV numbers will move — they are averages over a random partition — but the chosen degree should stay at 3. If it moves, that tells you the choice between two adjacent degrees is not supported by 25 rows, which is itself worth knowing.

8 Summary

1. “Closest” is a choice. On the five points the direct slope is 0.6000, the inverse slope 1.0000 and the orthogonal slope 0.7208; each minimises its own criterion and nothing else, and $b^{\text{dir}}/b^{\text{inv}} = r^2 = 0.6$.
2. **Assume $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$ independently, write the likelihood, take logarithms, and $\ell = \text{const} - \text{SSE}/(2\sigma^2)$. Maximising ℓ is identical to minimising the SSE.** Least squares is the Gaussian maximum likelihood estimator.
3. σ^2 is a positive multiplier of $-\text{SSE}$ and cannot move the estimate: the same $b_1 = 0.60000$ maximised the likelihood at $\sigma^2 = 0.48$ and at $\sigma^2 = 1$.
4. $\hat{\sigma}_{\text{ML}}^2 = \text{SSE}/n$ is **biased downwards** by the factor $(n-2)/n$. Over 20,000 datasets with $\sigma^2 = 4$ and $n = 10$ it averaged 3.2117 against a predicted 3.2000, and was too small in 73.4% of runs. Use $s^2 = \text{SSE}/(n-2)$.
5. Change the noise to Laplace and the MLE becomes least *absolute* deviations, which recovered the true intercept to 0.23 against least squares’ 1.09 on contaminated data.
6. $R^2 = 1 - \text{SS}_{\text{res}}/\text{SS}_{\text{tot}} = 1 - 2.40/6.00 = 0.6000$ on the five points, equal to r^2 and to $b_1^2 S_{xx}/S_{yy}$, and $s = 0.8944$.
7. R^2 **never falls when you add a predictor**. Twenty-seven pure-noise columns on 30 rows took it from 0.2413 to 0.9401, and the smallest of the 27 steps was $+3 \times 10^{-6}$.
8. Adjusted R^2 caught that (-0.7375) , but on 442 rows with 410 junk columns it read $+0.6597$ while cross-validation read -5.7605 . It is a penalty, not a test.
9. Held-out R^2 has **no lower bound**: -3 on three numbers by hand, -1.106 for a degree-14 fit, -22.04 for a degree-3 polynomial on diabetes against -0.0001 for predicting the mean.
10. Anscombe’s four datasets share every summary statistic and $R^2 = 0.666$. The residual diagnostics separate them completely: curvature $R^2 = 1.0000$ for II, largest residual $2.80 \times$ the next for III, leverage 1.0000 and an undefined deleted slope for IV.
11. Heteroscedastic data gave $R^2 = 0.9267$ with a residual sd ratio of 1.82 and Breusch–Pagan $p \approx 2 \times 10^{-6}$. Coefficients survive it; every standard error and p -value does not. $\log y$ took

the statistic from 22.91 to 3.24 ($p = 0.072$) — most of the way, not all of it.

12. Over-fitting: training RMSE fell at all 20 steps while test RMSE turned up after **degree 3**, reaching 153 at degree 20 — $548\times$ worse. At $n = 1000$ the same degree 20 costs 0.0036 in RMSE, 1.4%.
13. **5-fold CV and LOOCV both chose degree 3, the same answer as the test set, using only the training rows**, while the training score chose the largest model offered. A single validation split chose 3, 4 or 5 depending on the seed, and estimated the degree-3 error anywhere between 0.2243 and 0.4415.

9 Practice questions

Attempt these before reading Section 10. Questions 1, 2, 3, 4, 5 and 6 are pure hand work and are the ones most likely to appear on a paper.

1. Four points: $(1, 1)$, $(2, 3)$, $(3, 2)$, $(4, 4)$. (a) Compute $\bar{x}$, $\bar{y}$, S_{xx} , S_{yy} and S_{xy} . (b) Give the direct regression slope, the inverse regression slope expressed as y against x , and the orthogonal slope. (c) Verify that the ratio of the first two is r^2 . (d) Which line would you use to predict y from x , and why?
2. Starting from a blank page, derive the maximum likelihood estimator of (β_0, β_1) for simple linear regression. State the model and its assumptions, write the likelihood, take logarithms, and show *explicitly* why the answer does not depend on σ^2 . Then state what changes if the noise is Laplace rather than Gaussian.
3. A regression on $n = 6$ points gives $\text{SSE} = 12$. (a) Give $\hat{\sigma}_{\text{ML}}^2$ and the unbiased s^2 . (b) If the true σ^2 is 3, what is $\mathbb{E}[\hat{\sigma}_{\text{ML}}^2]$, and by what percentage does the MLE understate the truth on average? (c) At what n does that understatement fall below 1%?
4. Data: $x = (0, 1, 2, 3, 4)$, $y = (1, 3, 2, 5, 4)$. (a) Fit the direct regression line by hand. (b) Tabulate the fitted values and residuals and compute SS_{tot} , SS_{res} and SS_{reg} . (c) Give R^2 and check it two other ways. (d) Give s , the standard error of the estimate.
5. A model with $n = 20$ and one predictor has $R^2 = 0.60$. You add eight more predictors and R^2 rises to 0.68. (a) Compute adjusted R^2 for both models. (b) Which model does adjusted R^2 prefer? (c) Holding R^2 fixed at 0.68, at what p does adjusted R^2 first become negative? (d) Explain in one sentence why R^2 rose, without assuming the new predictors are useful.
6. On three held-out points $y = (4, 6, 8)$ your model predicts $(10, 6, 2)$. (a) Compute R^2 . (b) What constant prediction would give $R^2 = 0$, and what would the constant 5 give? (c) Explain in two sentences why R^2 is bounded below by 0 on training data and not on held-out data.
7. A colleague reports: “I fitted a degree-3 polynomial expansion of the ten diabetes features and got $R^2 = 0.918$. This is much better than the plain linear model’s 0.555, so we should use it.” Their notebook, in order: (i) load; (ii) build the degree-3 features; (iii) fit on all 331 training rows; (iv) print `model.score(X_train, y_train)`. Identify every problem, quantify the damage using measured numbers from these notes, and write the corrected four lines of code.
8. You fit a model and get $R^2 = 0.89$. The residuals against fitted values form a clear wedge, narrow on the left and wide on the right. (a) Name the violated assumption. (b) State precisely what is and is not invalidated. (c) Give a test statistic you could compute and how. (d) Give one remedy and the measured effect it had in Section 5.4.

9. From the degree table of Section 6.2: (a) Which degree does each of “lowest training RMSE”, “lowest 5-fold CV RMSE” and “lowest test RMSE” select? (b) Prove that training RMSE cannot increase with degree. (c) By what percentage does training RMSE improve from degree 3 to degree 20, and by what factor does test RMSE worsen? (d) Explain the apparent contradiction in one sentence.
10. You are given 60 rows and one predictor, and asked to choose a polynomial degree between 1 and 10 and then report an honest estimate of how well the final model will do on new data. Write the protocol as a numbered list. For each step say what data it may use, and cite one measured number from these notes justifying the step.

10 Solutions

1. (a) Both means are 2.5. The x -deviations are $-1.5, -0.5, 0.5, 1.5$ and the y -deviations $-1.5, 0.5, -0.5, 1.5$, so $S_{xx} = 2.25 + 0.25 + 0.25 + 2.25 = 5$, likewise $S_{yy} = 5$, and $S_{xy} = 2.25 - 0.25 - 0.25 + 2.25 = 4$.

(b) Direct $= S_{xy}/S_{xx} = 4/5 = 0.8$. Inverse, as y against x , $= S_{yy}/S_{xy} = 5/4 = 1.25$. For the orthogonal slope the covariance matrix is $\frac{1}{3} \begin{pmatrix} 5 & 4 \\ 4 & 5 \end{pmatrix}$, which has equal diagonal entries, so its eigenvectors are $(1, 1)/\sqrt{2}$ and $(1, -1)/\sqrt{2}$ and the leading slope is exactly 1.

(c) $0.8/1.25 = 0.64$, and $r^2 = S_{xy}^2/(S_{xx}S_{yy}) = 16/25 = 0.64$. ✓

(d) The direct line, because the task named is predicting y from x . Note that all three slopes are different and $0.8 < 1.0 < 1.25$.

2. *Model.* $y_i = \beta_0 + \beta_1 x_i + \varepsilon_i$ with $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$, independent, $i = 1, \dots, n$; the x_i are fixed and known. Assumptions: linear conditional mean, constant variance, independence, normality.

Likelihood. With $e_i = y_i - \beta_0 - \beta_1 x_i$,

$$L = \prod_{i=1}^n (2\pi\sigma^2)^{-1/2} \exp\left(-\frac{e_i^2}{2\sigma^2}\right) = (2\pi\sigma^2)^{-n/2} \exp\left(-\frac{1}{2\sigma^2} \sum_i e_i^2\right).$$

Log-likelihood. $\ell = -\frac{n}{2} \log(2\pi) - \frac{n}{2} \log \sigma^2 - \frac{1}{2\sigma^2} \sum_i e_i^2$.

Why σ^2 drops out. Only the third term contains β_0 and β_1 . Write $\ell = A - cS(\beta_0, \beta_1)$ where $A = -\frac{n}{2} \log(2\pi) - \frac{n}{2} \log \sigma^2$ is free of the β 's and $c = 1/(2\sigma^2) > 0$. Maximising $A - cS$ over (β_0, β_1) is minimising S , and the value of c changes the *height* of the objective but not its *argmax*, because c is a strictly positive constant. Hence the ML estimates are the least squares estimates for every $\sigma^2 > 0$. Setting $\partial S/\partial \beta_0 = \partial S/\partial \beta_1 = 0$ gives the normal equations and $\hat{\beta}_1 = S_{xy}/S_{xx}$, $\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$.

Laplace noise. The density is $\frac{1}{2b} e^{-|e|/b}$, so $\ell = -n \log(2b) - \frac{1}{b} \sum_i |e_i|$ and the MLE minimises $\sum_i |e_i|$: least absolute deviations. Measured in Section 3.9, the LAD fit recovered the true intercept 2.00 as 2.77 where least squares gave 3.00; the two slopes were indistinguishable on that single sample, which is itself worth noting.

3. (a) $\hat{\sigma}_{\text{ML}}^2 = 12/6 = 2.0$ and $s^2 = 12/4 = 3.0$. (b) $\mathbb{E}[\hat{\sigma}_{\text{ML}}^2] = \frac{n-2}{n} \sigma^2 = \frac{4}{6} \times 3 = 2.0$; the understatement is $2/n = 33.33\%$. (c) $2/n < 0.01$ requires $n > 200$, so $n \geq 201$.

```
SSE 12.0  n 6
sigma^2 MLE      = 12.0/6 = 2.0000
sigma^2 unbiased = 12.0/4 = 3.0000
E[MLE] = (n-2)/n * sigma^2 = 0.6667 * 3.0 = 2.0000
understated by 33.33%
```

Note that here the unbiased estimate happens to equal the true σ^2 exactly; that is a coincidence of the chosen SSE, not a property.

4. (a) $\bar{x} = 2$, $\bar{y} = 3$. The x -deviations are $-2, -1, 0, 1, 2$ and the y -deviations $-2, 0, -1, 2, 1$, so $S_{xx} = 10$, $S_{yy} = 10$ and $S_{xy} = 4 + 0 + 0 + 2 + 2 = 8$. Hence $b_1 = 8/10 = 0.8$ and $b_0 = 3 - 0.8 \times 2 = 1.4$.

(b) Fitted $(1.4, 2.2, 3.0, 3.8, 4.6)$; residuals $(-0.4, 0.8, -1.0, 1.2, -0.6)$. $SS_{\text{tot}} = 10$, $SS_{\text{res}} = 0.16 + 0.64 + 1.00 + 1.44 + 0.36 = 3.60$, $SS_{\text{reg}} = 6.40$.

(c) $R^2 = 1 - 3.6/10 = 0.64$. Check one: $SS_{\text{reg}}/SS_{\text{tot}} = 6.4/10 = 0.64$, and independently $b_1^2 S_{xx} = 0.64 \times 10 = 6.40$. Check two: $r^2 = 8^2/(10 \times 10) = 0.64$.

(d) $s = \sqrt{3.6/(5-2)} = \sqrt{1.2} = 1.0954$.

```
xbar 2.0 ybar 3.0 Sxx 10.0 Syy 10.0 Sxy 8.0
b1 0.8000 b0 1.4000
fitted [1.4 2.2 3. 3.8 4.6]
residuals [-0.4 0.8 -1. 1.2 -0.6]
SSres 3.6000 SStot 10.0000 SSreg 6.4000 b1^2*Sxx 6.4000
R2 0.6400 r^2 0.6400 s 1.0954
```

5. (a) $p = 1$: $1 - 0.40 \times \frac{19}{18} = 1 - 0.4222 = 0.5778$. $p = 9$: $1 - 0.32 \times \frac{19}{10} = 1 - 0.608 = 0.3920$.

(b) The first, clearly: adjusted R^2 fell by 0.19 even though R^2 rose by 0.08. Eight predictors bought 0.08 of R^2 , and eight degrees of freedom cost far more than that.

(c) Solve $1 - 0.32 \times \frac{19}{19-p} < 0$, i.e. $6.08 > 19 - p$, i.e. $p > 12.92$. So $p = 13$ is the first negative value, at -0.0133 .

(d) Because R^2 cannot fall when a predictor is added: the previous fit remains available with the new coefficient set to zero, so the minimised SS_{res} can only decrease. A rise in R^2 is therefore evidence of nothing.

```
n=20 p=1 R2 0.6000 adjusted 0.5778
n=20 p=9 R2 0.6800 adjusted 0.3920
n=20 p=12 R2 0.6800 adjusted +0.1314
n=20 p=13 R2 0.6800 adjusted -0.0133
n=20 p=14 R2 0.6800 adjusted -0.2160
```

6. (a) $\bar{y}_{\text{test}} = 6$; $SS_{\text{tot}} = 4 + 0 + 4 = 8$; $SS_{\text{res}} = 36 + 0 + 36 = 72$; $R^2 = 1 - 72/8 = -8$.

(b) The constant 6 — the test mean — gives exactly 0. The constant 5 gives $SS_{\text{res}} = 1 + 1 + 9 = 11$ and $R^2 = 1 - 11/8 = -0.375$: even the wrong constant beats the model.

(c) On training data the intercept is fitted, so $\hat{y} = \bar{y}$ is one of the candidate fits and least squares cannot do worse than it; therefore $SS_{\text{res}} \leq SS_{\text{tot}}$ and $R^2 \geq 0$. On held-out data $\bar{y}_{\text{test}}$ was never available to the fitting procedure, so nothing constrains SS_{res} relative to SS_{tot} and R^2 is unbounded below.

```
SStot 8.0 SSres 72.0 R2 -8.0000
R2 of the constant 6.0 0.0000
R2 of the constant 5.0 -0.3750
```

7. Three problems, of increasing seriousness.

One — the number reported is a training score. `model.score(X_train, y_train)` is R^2 on the rows the model was fitted to. It is not an estimate of anything about future data.

Two — the comparison is between models of different sizes, using a metric that cannot make that comparison. Degree-3 expansion of ten features gives 285 predictors against the linear model's 10. R^2 cannot decrease when predictors are added (Section 4.4), so $0.918 > 0.555$ was guaranteed

before anyone looked at the data. The measured demonstration: 27 columns of *pure noise* raised R^2 from 0.2413 to 0.9401 on 30 rows.

Three — and this is the one that matters — the model is catastrophically over-fitted. 285 parameters on 331 rows. From Section 6.4: 5-fold CV $R^2 = -1212.18$ and held-out $R^2 = -22.04$, against -0.0001 for predicting the training mean. The proposal is to replace a model scoring $+0.36$ on held-out data with one scoring -22 .

A fourth point worth making: adjusted R^2 would not have saved them. On diabetes with 410 junk columns it read $+0.6597$ while cross-validation read -5.7605 (Section 4.5).

Corrected code.

```
pipe = make_pipeline(PolynomialFeatures(deg, include_bias=False),
                    LinearRegression())
print(cross_val_score(pipe, X_train, y_train,
                    cv=KFold(5, shuffle=True, random_state=0),
                    scoring="r2").mean()) # choose deg by THIS
pipe.fit(X_train, y_train)
print(pipe.score(X_test, y_test)) # report once, at the end
```

model	train R2	5-fold CV R2	held-out R2	params
linear (10 features)	+0.5554	+0.5224	+0.3594	10
degree-2 polynomial	+0.6469	+0.3827	+0.2440	65
degree-3 polynomial	+0.9179	-1212.1828	-22.0428	285

8. (a) Constant variance — homoscedasticity. The data is heteroscedastic, with σ growing with $\hat{y}$.

(b) The coefficient estimates remain *unbiased*: least squares needs no constant-variance assumption for that, and by Gauss–Markov it is only the efficiency that is lost. What is invalidated is everything computed from a single σ : the standard errors of the coefficients, every t and F statistic, every p -value, and both confidence and prediction intervals. A prediction interval will be too wide at small $\hat{y}$ and far too narrow at large $\hat{y}$, which is the dangerous direction.

(c) The Breusch–Pagan test: regress the squared residuals on the fitted values, take nR^2 from that auxiliary regression, and compare it to a χ^2 with one degree of freedom. In Section 5.4 this gave 22.9065 with $p \approx 2 \times 10^{-6}$.

(d) Transform the response. Taking $\log y$ moved the Breusch–Pagan statistic from 22.9065 to 3.2426 ($p = 0.0717$) and the ratio of upper-half to lower-half residual sd from 1.82 to 0.89. Note that $p = 0.072$ is not a clean pass: the transform removed most of the trend in the variance but not all of it. Weighted least squares is the alternative when a transform is not acceptable, or not enough.

9. (a) Lowest training RMSE: **degree 20** (or 15 if the sweep stops there) — always the largest offered. Lowest 5-fold CV RMSE: **degree 3**. Lowest test RMSE: **degree 3**.

(b) The set of degree- d polynomials is a subset of the set of degree- $(d+1)$ polynomials — take the degree- $(d+1)$ coefficient to be zero. Least squares returns the minimum of the training SSE over the set it is given, and the minimum over a superset cannot exceed the minimum over a subset. Hence training SSE, and therefore training RMSE, is non-increasing in d . This was verified: the largest single-step change over 20 steps was -3.29×10^{-5} .

(c) Training RMSE fell from 0.2441 to 0.1466, an improvement of **39.9%**. Test RMSE rose from 0.2795 to 153.1554, worse by a factor of **547.9**.

(d) There is no contradiction: the extra flexibility was spent fitting the particular noise realisation in those 25 rows, which is real structure in the training data and pure interference in any other data.

10. A defensible protocol.

1. **Split first.** Hold out, say, 15 of the 60 rows as a test set and put them away. *May use:* nothing but a random number generator. *Why:* any decision made with these rows makes the final number a training score. In Section 7.2 the training score chose the largest model on offer.
2. **Cross-validate inside the remaining 45.** For each degree 1..10, run 5-fold CV and record the mean RMSE. *May use:* the 45 training rows only. *Why:* CV chose degree 3 — the same answer as the test set — using nothing but training rows.
3. **Do not use a single validation split.** *Why:* ten different 60/40 splits of the same 25 rows chose degrees 3, 4 and 5, and estimated the degree-3 error anywhere between 0.2243 and 0.4415 (Section 7.3).
4. **Wrap every fitted step in a Pipeline** so that the polynomial expansion and any scaling are refitted inside each fold. *Why:* anything the model learns from all 45 rows before the folds are cut is no longer held out.
5. **Prefer the simpler model on a tie.** If two degrees are within one CV standard error, take the smaller. *Why:* at $n = 15$ degree 20 produced a test RMSE of 53,293; complexity is cheap to add and expensive when wrong (Section 6.3).
6. **Refit the chosen degree on all 45 rows**, then score once on the 15 test rows and report that number. *Why:* it is the only number in the whole procedure computed on data no decision used.
7. **Report the residual plot with it.** *Why:* four datasets with $R^2 = 0.666$ had four different problems (Section 5.3).
8. **Say n when you report the degree.** *Why:* the same degree-20 model was catastrophic at $n = 15$ and cost 0.0036 at $n = 1000$.

11 References and further reading

All links were checked on 22 August 2026. Free resources are marked [free].

Textbooks

- Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006. **The prescribed text for this lecture.** §1.2.5 “Curve fitting re-visited” — the maximum likelihood derivation of Section 3.4, done for polynomial regression; §3.1.1 “Maximum likelihood and least squares”, the same argument in matrix form with the precision parameter $\beta = 1/\sigma^2$; §1.1 and §1.3 for the over-fitting figures and model selection; §3.2 the bias–variance decomposition.
- James, Witten, Hastie, Tibshirani and Taylor, *An Introduction to Statistical Learning with Python*. [free] §3.1.3 “Assessing the accuracy of the model” — RSE, R^2 , and the identity $R^2 = r^2$ for simple regression; §3.3.3 “Potential problems”, which is Section 5 of these notes in six numbered headings; §5.1 the validation set approach, LOOCV and k -fold cross-validation, and §5.1.4 on the bias–variance trade-off for k . <https://www.statlearning.com/>
- Müller and Guido, *Introduction to Machine Learning with Python*, O’Reilly, 2016. §5.1 cross-validation — §5.1.1 in scikit-learn, §5.1.2 the benefits, §5.1.3 stratified and other splitters; §2.3.3 on linear models and the training/test gap. Notebooks [free]: https://github.com/amueller/introduction_to_ml_with_python

- Deisenroth, Faisal and Ong, *Mathematics for Machine Learning*. [free] Ch. 9, “Linear Regression”: §9.2 parameter estimation, which does maximum likelihood *first* and least squares as its consequence — the order used in these notes; §9.2.2 on overfitting in linear regression. <https://mml-book.github.io/>
- Murphy, *Probabilistic Machine Learning: An Introduction*. [free] §11.2 “Least squares linear regression”, including §11.2.3 on the MLE for σ^2 and why it is biased, and §11.4 on robust linear regression — the Laplace likelihood of Section 3.9, worked out properly. <https://probml.github.io/pml-book/book1.html>
- Hastie, Tibshirani and Friedman, *The Elements of Statistical Learning*, 2e, Springer. [free] §7.2 bias, variance and model complexity — the theory behind Figure 13; §7.10 cross-validation, including the choice of k and the one-standard-error rule. <https://hastie.su.domains/ElemStatLearn/>

Online courses

- **NPTEL, Linear Regression Analysis and Forecasting** — Prof. Shalabh, IIT Kanpur. The closest match anywhere to the vocabulary this examination uses: the direct, inverse and orthogonal regression methods of Section 2 are named and derived exactly as they are here, and the maximum likelihood estimation of the simple linear model is a lecture of its own. [free] <https://nptel.ac.in/courses/111104098>
- **NPTEL, Introduction to Machine Learning** — Prof. Balaraman Ravindran, IIT Madras; the reference course for the semester. The linear regression week covers least squares and the bias–variance trade-off. [free] <https://nptel.ac.in/courses/106106139>
- **SWAYAM** — the national portal hosting NPTEL, with the enrolment and certification route. [free] <https://swayam.gov.in/>
- **Google Machine Learning Crash Course** — the “Linear regression”, “Overfitting” and “Datasets, generalization” modules; short, browser-based, and a good first pass before the mathematics. [free] <https://developers.google.com/machine-learning/crash-course>

Videos

- StatQuest with Josh Starmer, *Maximum Likelihood, clearly explained!!!* — the likelihood-versus-probability distinction of Section 3 at a whiteboard, in six minutes. Watch this first if the word “likelihood” has not yet clicked. [free] <https://youtu.be/XepXt19YKwc>
- StatQuest with Josh Starmer, *R-squared, Clearly Explained!!!* — SS_{tot} , SS_{res} and the ratio, drawn rather than derived. Covers the “ R^2 always goes up” point of Section 4.4. [free] <https://youtu.be/2AQKmw14mHM>
- StatQuest with Josh Starmer, *Machine Learning Fundamentals: Cross Validation* — k -fold and leave-one-out in five minutes, with the “do not touch the test set” rule stated plainly. [free] <https://youtu.be/fSytzGwwBVw>
- StatQuest with Josh Starmer, *Machine Learning Fundamentals: Bias and Variance* — the theory underneath Figure 13: why the test curve has to turn. [free] <https://youtu.be/EuB Bz3bI-aA>
- CampusX, *Regression Metrics | MSE, MAE & RMSE | R2 Score & Adjusted R2 Score* — the whole of Section 4 in examination vocabulary, including the adjusted R^2 formula and when it goes negative. [free] <https://youtu.be/Ti7c-Hz7GSM>

- Mahesh Huddar, *Linear Regression Numerical Example with one Independent Variable* — a full hand calculation of the kind in Section 4.2, at the speed a written paper requires. [free] <https://youtu.be/OXMATSKULRY>

Documentation and interactive explainers

- Seeing Theory (Brown University), *Chapter 6: Regression Analysis* — drag the points of **Anscombe’s quartet** and watch the least-squares line and the SSE move. The best five minutes you can spend on Section 5.2. [free] <https://seeing-theory.brown.edu/regression-analysis/index.html>
- MLU-Explain, *Linear Regression* — Jared Wilber’s scrollable explainer covering least squares, R^2 and the closed form, with every figure interactive. [free] <https://mlu-explain.github.io/linear-regression/>
- scikit-learn user guide: *Metrics and scoring* (§3.4, with the exact definition of `r2_score` and the note that it can be negative); *Cross-validation: evaluating estimator performance* (§3.1, with `KFold`, `LeaveOneOut` and the Pipeline warning). [free] https://scikit-learn.org/stable/modules/model_evaluation.html
https://scikit-learn.org/stable/modules/cross_validation.html
- scikit-learn example, *Underfitting vs. Overfitting* — the figure of Section 6.2 as a runnable script, using the same cosine target and polynomial degrees. [free] https://scikit-learn.org/stable/auto_examples/model_selection/plot_underfitting_overfitting.html

Articles

- F. J. Anscombe, “Graphs in Statistical Analysis”, *The American Statistician* **27**(1), 1973, pp. 17–21. The four datasets of Section 5.2, and the sentence they exist to support: “*make both calculations and graphs. Both sorts of output should be studied; each will contribute to understanding.*”
- Matejka and Fitzmaurice, “Same Stats, Different Graphs”, ACM CHI 2017 — how to *generate* Anscombe-like datasets, including the Datasaurus. Convincing proof that summary statistics do not determine a dataset. [free] <https://www.research.autodesk.com/publications/same-stats-different-graphs/>

Sources: James et al., *An Introduction to Statistical Learning with Python*, Ch. 3–6.; Bishop, *Pattern Recognition and Machine Learning*, Ch. 3.; Deisenroth, Faisal and Ong, *Mathematics for Machine Learning*, Ch. 9.; Müller and Guido, *Introduction to Machine Learning with Python*, §2.3.